
Table of Contents

P5: Oleart y Chen	1
n = 3	1
n = 6	2
n = 9	3
Apartado c	4
Apartado d	8
-----FUNCIONES-----	10

P5: Oleart y Chen

`%%Apartado a`

```
n = 5;  
m = 600;
```

`%Hacer el vector de nodos`

```
z = generar_puntos(m);
```

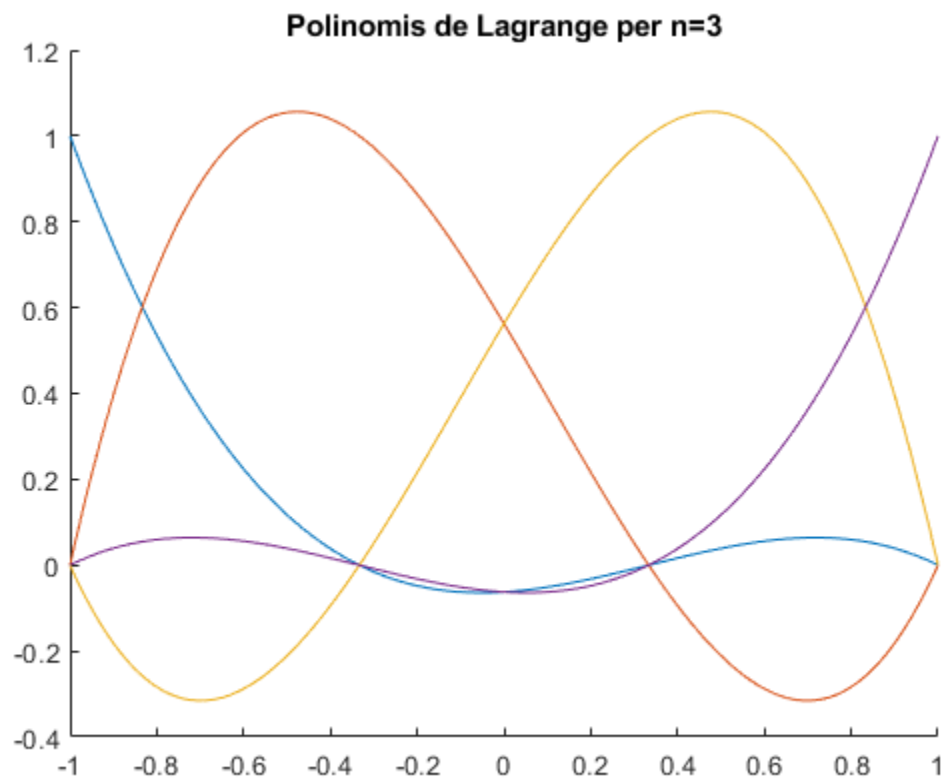
`%Calcular la matriz`

```
P = mat_pol(n, m);
```

`%%Apartado b`

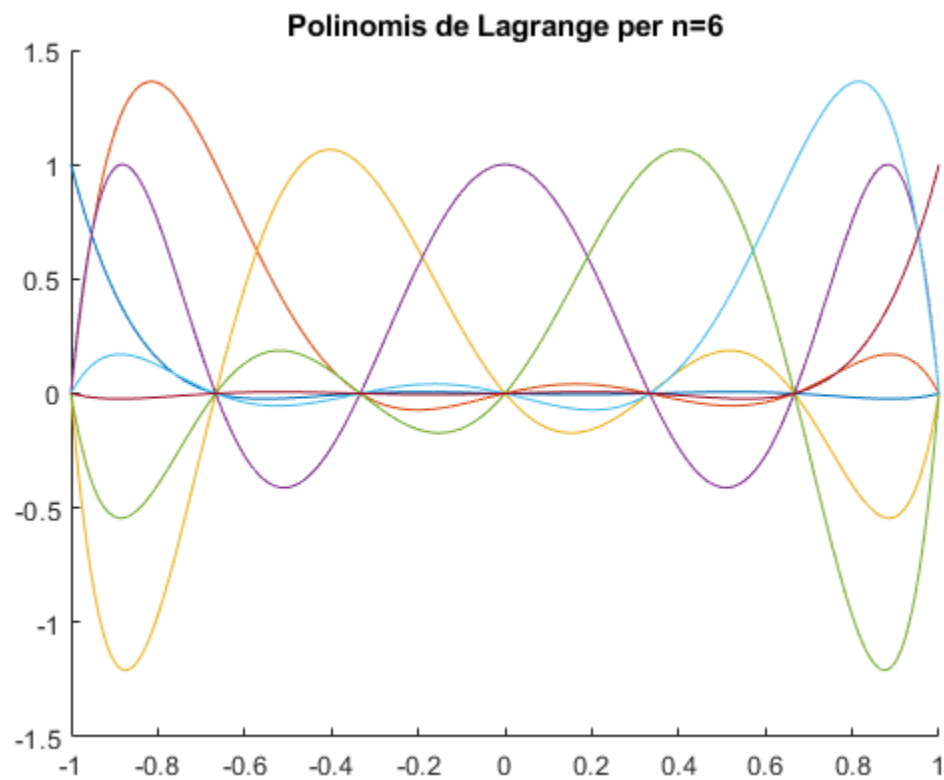
n = 3

```
plot_lag(3, m);  
title('Polinomis de Lagrange per n=3')
```



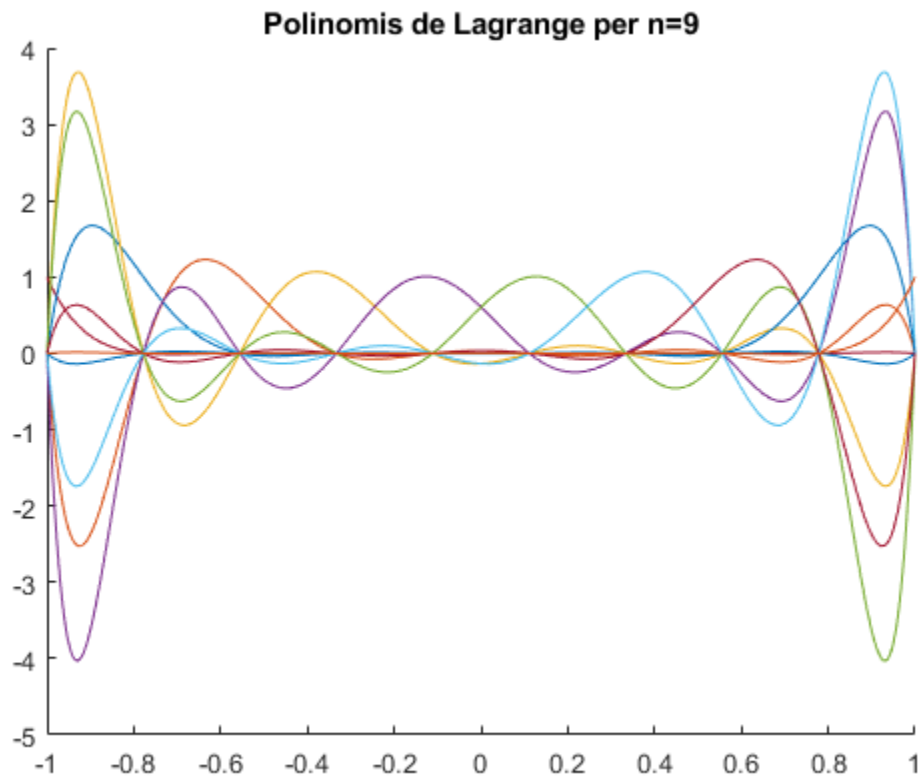
n = 6

```
figure(2);  
plot_lag(6, m);  
title('Polinomis de Lagrange per n=6')
```



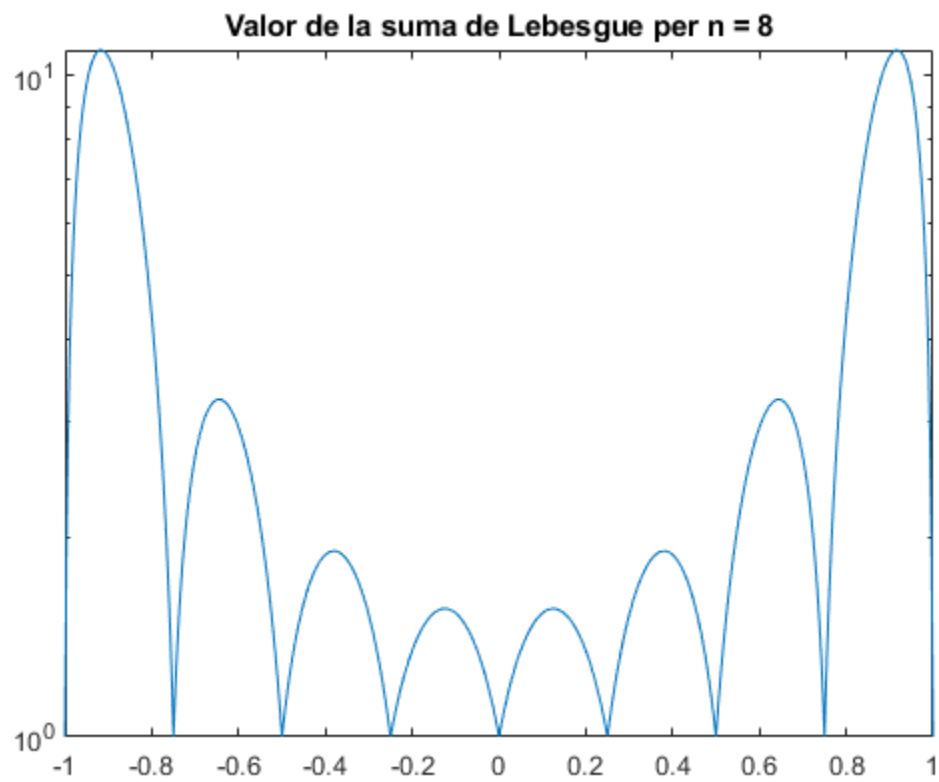
n = 9

```
figure(3);  
plot_lag(9, m);  
title('Polinomis de Lagrange per n=9')
```

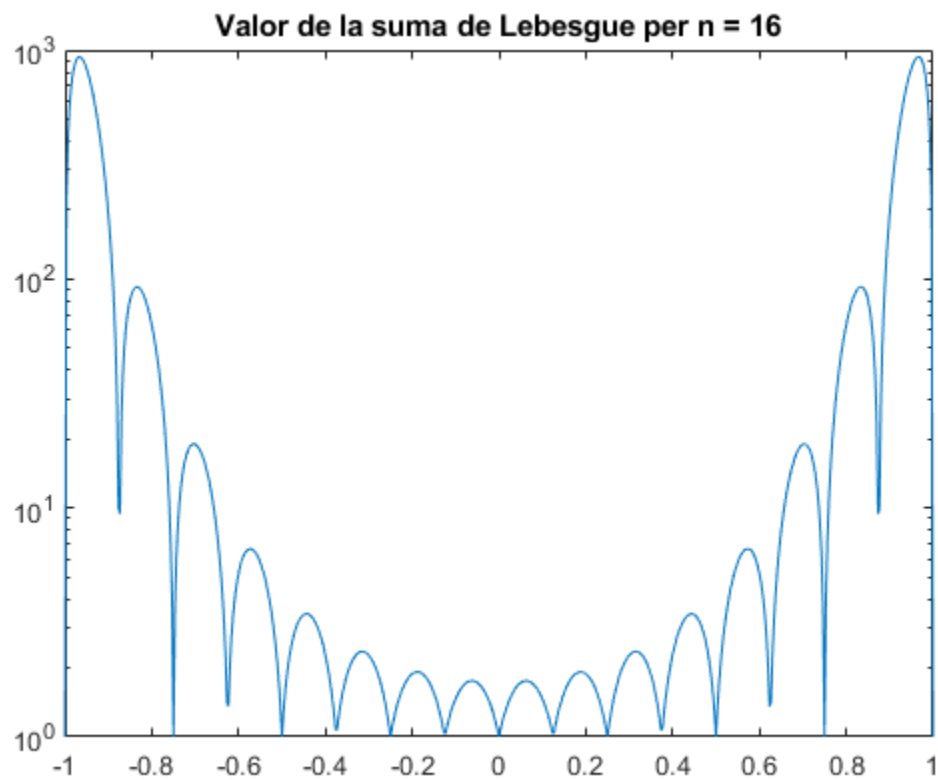


Apartado c

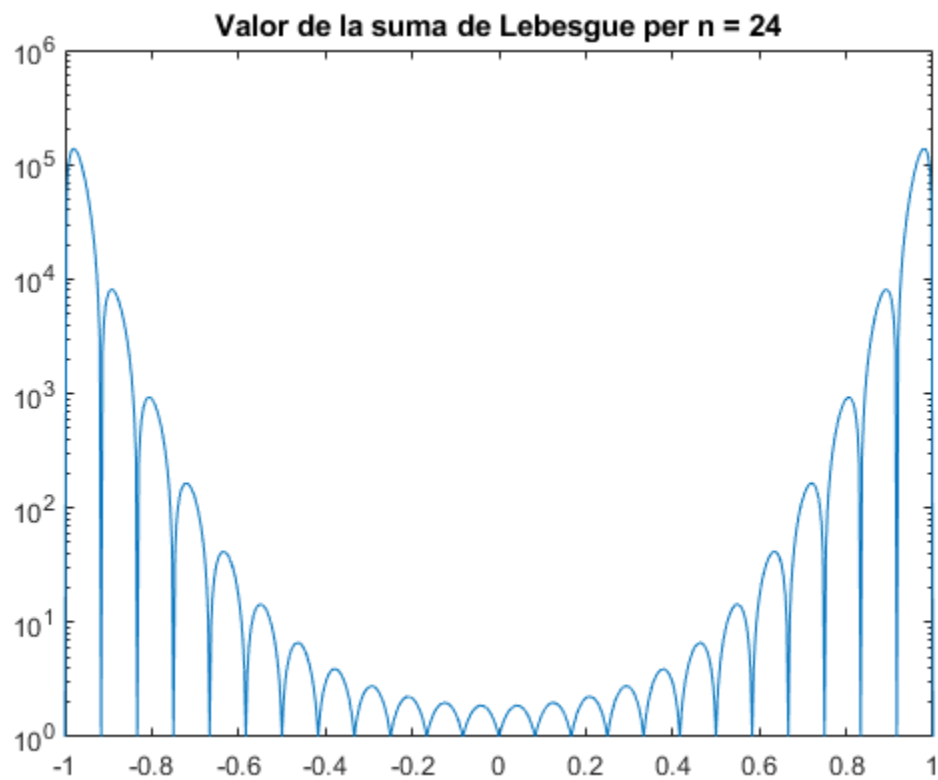
```
n = 8;  
P_8 = mat_pol(n, m);  
leb_8 = Lebesgue(P_8);  
figure(4);  
semilogy(z, leb_8);  
title('Valor de la suma de Lebesgue per n = 8')
```



```
n = 16;  
P_16 = mat_pol(n, m);  
leb_16 = Lebesgue(P_16);  
figure(5);  
semilogy(z, leb_16);  
title('Valor de la suma de Lebesgue per n = 16')
```



```
n = 24;  
P_24 = mat_pol(n, m);  
figure(6);  
leb_24 = Lebesgue(P_24);  
semilogy(z, leb_24);  
title('Valor de la suma de Lebesgue per n = 24')
```

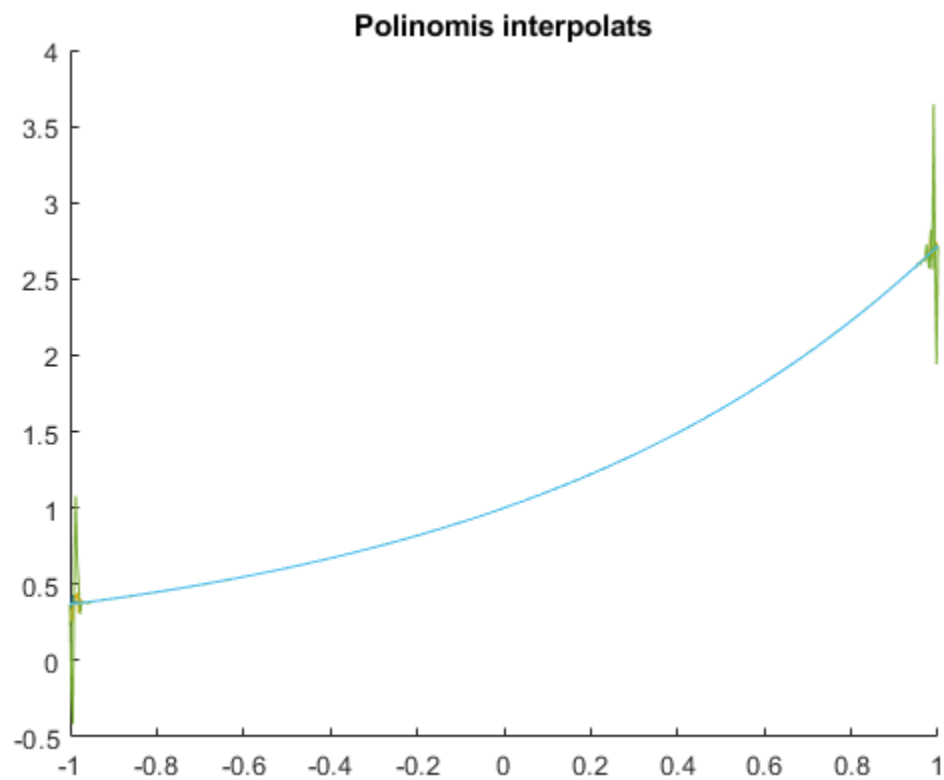


```
n = 32;  
P_32 = mat_pol(n, m);  
figure(7);  
leb_32 = Lebesgue(P_32);  
semilogy(z, leb_32);  
title('Valor de la suma de Lebesgue per n = 32')
```

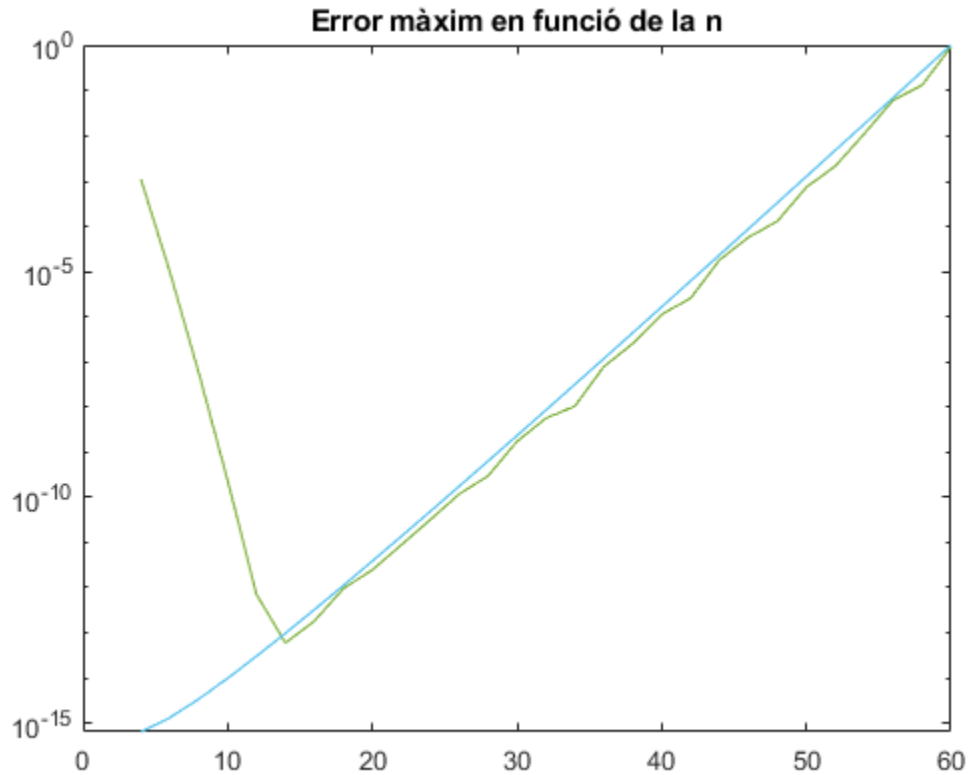


Apartado d

```
en = [];  
figure(8);  
for n = 4:2:60  
    pol_int= interpolar(n, m, @f);  
    hold on;  
    plot(z, pol_int); %polinomi interpolador  
    plot(z, f(z)); %funcio real  
    hold off;  
    %calcul de l error  
  
    en = [en, max(abs(pol_int' - exp(z)))];  
end  
title('Polinomis interpolats')
```



```
figure(9);  
n1 = (4:2:60);  
epsil = 2.^n1./(n1.*log(n1));  
semilogy(n1, en);  
hold on;  
semilogy(n1, epsil*eps);  
title('Error màxim en funció de la n')
```



-----FUNCIONES-----

```
function ptos = generar_puntos(n)
    %ptos = zeros(n+1);
    ptos = [];
    for j = 0:n
        % ptos(j+1) = -1 + (2*j)/n;
        ptos = [ptos, -1 + (2*j)/n];
    end
end

%Dado el vector de nodos x, el punto z, el indice del cardinal i y el
%numero total de cardiales n, devuelve el polinomio cardinal de Lagrange
%correspondiente
function lag_card = lag_card(x, z, k, n)
    lag_card = 1;
    for j = 1: n+1
        if j ~= k
            lag_card = lag_card * ((z - x(j)) / (x(k) - x(j)));
        end
    end
end

function M = mat_pol(n, m)
```

```

x = generar_puntos(n);
z = generar_puntos(m);
M = zeros(m+1, n+1);
for j=1:m+1
    for k=1:n+1
        M(j,k)= lag_card(x, z(j), k, n);
    end
end
end

function plot_lag(n, m)
    z = generar_puntos(m);
    P = mat_pol(n, m);
    hold on;
    for j = 1:n+1
        plot(z, P(:,j));
    end
    hold off;
end

function leb = Lebesgue(P)
    [~, n] = size(P);
    mat = ones(n);
    vect = mat(:,n);

    leb_v = abs(P) * vect; %si multiplicamos la matriz por un vector
    leb = leb_v';          % de unos, ya devuelve la suma
end

%retorna el polinomi interpolat
function pol = interpolar(n, m, f)
    P = mat_pol(n, m);
    x = generar_puntos(n);
    pol = P * f(x)';
end

function f = f(x)
    f = exp(x);
end

```

Published with MATLAB® R2024a